Technical University of Crete
School of Electrical and Computer Engineering
Course: **Optimization**
Exercise 3
Report Submission Date: December 8, 2025
Instructor: Athanasios P. Liavas
150/1000 points

In the first part of this exercise, we shall solve an unconstrained convex optimization problem using the gradient descent and the Newton methods.

(70) Let $\mathbf{c} \in \mathbb{R}^n$, $\mathbf{b} \in \mathbb{R}^m$, and $\mathbf{A} \in \mathbb{R}^{m \times n}$, with rows $\mathbf{a}_i^T$, for $i = 1, \ldots, m$. Consider the function $f : \mathbf{dom} f \subseteq \mathbb{R}^n \to \mathbb{R}$, defined as

$$f(\mathbf{x}) = \mathbf{c}^T \mathbf{x} - \sum_{i=1}^{m} \log(b_i - \mathbf{a}_i^T \mathbf{x}) = \mathbf{c}^T \mathbf{x} - \mathtt{sum}\left(\mathtt{log}(\mathbf{b} - \mathbf{A}\mathbf{x})\right), \tag{1}$$

with $\mathbf{x} \in \mathbb{R}^n$ and $m > n$ (or $m \gg n$) (for example, $(n, m) = (2, 20), (50, 200)$. If you have patience and enough memory in the computer, you can set $(n, m) = (300, 800)$ or larger values).

Some observations are as follows (see Boyd–Vandenberghe, pages 141, 419–422, 458–459, 472, 492):

- The set $\mathbf{dom} f$ contains only the points $\mathbf{x} \in \mathbb{R}^n$ for which the arguments of the logarithms are positive.

  Prove that

  1. (5) the set $\mathbf{dom} f$ is convex.
  2. (5) Function f is convex.

- Observe that if $b_i > 0$, for $i = 1, \ldots, m$, then $\mathbf{dom} f \neq \emptyset$, because $\mathbf{x} = \mathbf{0}$ is a feasible point (in the experiments, it makes sense to *always* use this convention).

- Function f may be unbounded from below. In this case, the problem has no solution (no need to do something for it - `cvx` will help you to identify these cases).

- If a solution $\mathbf{x}_*$ exists, then it lies in the interior of $\mathbf{dom} f$, because when we approach the boundary of $\mathbf{dom} f$ the value of the function increases without bound. This means that a necessary and sufficient condition for $\mathbf{x}_*$ to be an optimal point is $\nabla f(\mathbf{x}_*) = \mathbf{0}$.

Our study will proceed as follows.

1. Create quantities $\mathbf{A}$, $\mathbf{b}$, and $\mathbf{c}$ randomly (in order to be certain that the problem is feasible, create $\mathbf{b}$ with non-negative elements).

2. Minimize f using the `cvx`. If the problem has a solution, then `cvx` will compute it, otherwise it will display a message saying that the problem has no solution.

3. (10) If $n = 2$, then plot f and its level sets in the neighborhood of the optimum point. You can check whether a point $\mathbf{x}$ belongs to the domain of f by checking the argument of the logarithm at this point. If a point $\mathbf{x}$ belongs to $\mathbf{dom} f$, then you can compute the value of f at this point. Otherwise, you can give an arbitrarily large value to f at this point (for example, $f(\mathbf{x}) = 10^3$).

4. (20) Assuming that $\mathbf{x} = \mathbf{0}$ is a feasible point, minimize f using the gradient algorithm with backtracking line search, starting from $\mathbf{x}_0 = \mathbf{0}$. Since $\mathbf{dom} f \subset \mathbb{R}^n$, the implementation will have the following main difference from the baseline implementation of the gradient descent algorithm.

   - In step $(k+1)$, given the vectors $\mathbf{x}_k$ and $\nabla f(\mathbf{x}_k)$, and having set $t = 1$, before starting the backtracking, you should check whether the point $\mathbf{x}_k + t\Delta\mathbf{x}_k$ belongs to $\mathbf{dom} f$ or not. If it does not belong to $\mathbf{dom} f$, then you must put $t := \beta t$ (β is the backtracking parameter) and repeat this process until you find a point that belongs to $\mathbf{dom} f$.[1] When you arrive at a point that belongs to $\mathbf{dom} f$, then you can proceed to the typical backtracking line search.

5. (20) Following the analogous procedure, minimize function f using the Newton algorithm.

6. (10) Plot, using `semilogy`, quantities $(f_{\text{gradient}}(\mathbf{x}_k) - p_*)$ and $(f_{\text{newton}}(\mathbf{x}_k) - p_*)$, as a function of step k. What do you observe for small and large values of the pair (n, m)?

[1] It may help you in the understanding of the behavior of the method if you print a message every time point $\mathbf{x} + t\Delta\mathbf{x}$ does not belong in $\mathbf{dom} f$. You may observe different behavior far away and close to the solution.

In the second part of this exercise, we shall solve simple (but important) optimization problems by explicitly solving the KKT conditions. In all cases, start with a drawing of the problem.

(80) The problems are the following:

1. (20) Compute the projection of $\mathbf{x}_0 \in \mathbb{R}^n$ onto the set $\mathbf{B}(\mathbf{0}, r) := \{\mathbf{x} \in \mathbb{R}^n \mid \|\mathbf{x}\|_2 \leq r\}$.

   (a) Draw a scheme of the problem.

   (b) Write down the optimization problem you must solve, in terms of **differentiable** functions.

   (c) Write down the KKT conditions, in terms of the optimal parameters $\mathbf{x}_*$ and λ_*.

   (d) Consider the case $\lambda_* > 0$. What is the conclusion?

   (e) Consider the case $\lambda_* = 0$. What is the conclusion?

2. (20) Let $\mathbf{a} \in \mathbb{R}^n$. Compute the projection of $\mathbf{x}_0 \in \mathbb{R}^n$ onto set $\mathbb{S} := \{\mathbf{x} \in \mathbb{R}^n \mid \mathbf{a} \leq \mathbf{x}\}$.

3. (20) Let $\mathbf{A} \in \mathbb{R}^{p \times n}$, $\mathbf{b} \in \mathbb{R}^p$, and $\mathbf{x}_0 \in \mathbb{R}^n$. Solve the problem

$$\min_{\mathbf{x}} \frac{1}{2} \|\mathbf{x}_0 - \mathbf{x}\|_2, \text{ subject to } \mathbf{A}\mathbf{x} = \mathbf{b}. \tag{2}$$

4. (20) Let $\mathbb{S} := \{\mathbf{x} \in \mathbb{R}^2 \mid \|\mathbf{x}\|_2 \leq 1, x_1 \geq 0, x_2 \geq 0, x_1 + x_2 \leq 1\}$. Solve the problem

$$\min_{\mathbf{x} \in \mathbb{S}} f_0(\mathbf{x}), \tag{3}$$

   with

   (a) $f_0(\mathbf{x}) = (x_1 - 2)^2 + (x_2 - 2)^2$,

   (b) $f_0(\mathbf{x}) = (x_1 + 2)^2 + (x_2 + 2)^2$.

   Draw a picture of $\mathbb{S}$ and check if there are any redundant constraints. Then, derive and solve the KKT.

SOLUTIONS

Patrikaki Aristea

1 PART A.

1. In MATLAB we first set the dimensions $\mathbf{n = 2}$ and $\mathbf{m = 20}$ and then generate the data that define the objective function:

```
A = rand(m,n);
b = rand(m,1) + 1.0;
c = randn(n,1);
```

   Note that b is generated with strictly positive entries (+ 1.0), which guarantees that x=0 satisfies $b - Ax > 0$.

   Definition in MATLAB of objective function and its gradient:

```
f     = @(x) c'*x - sum( log( b - A*x ) );
gradf = @(x) c + A' * ( 1 ./ (b - A*x) );
```

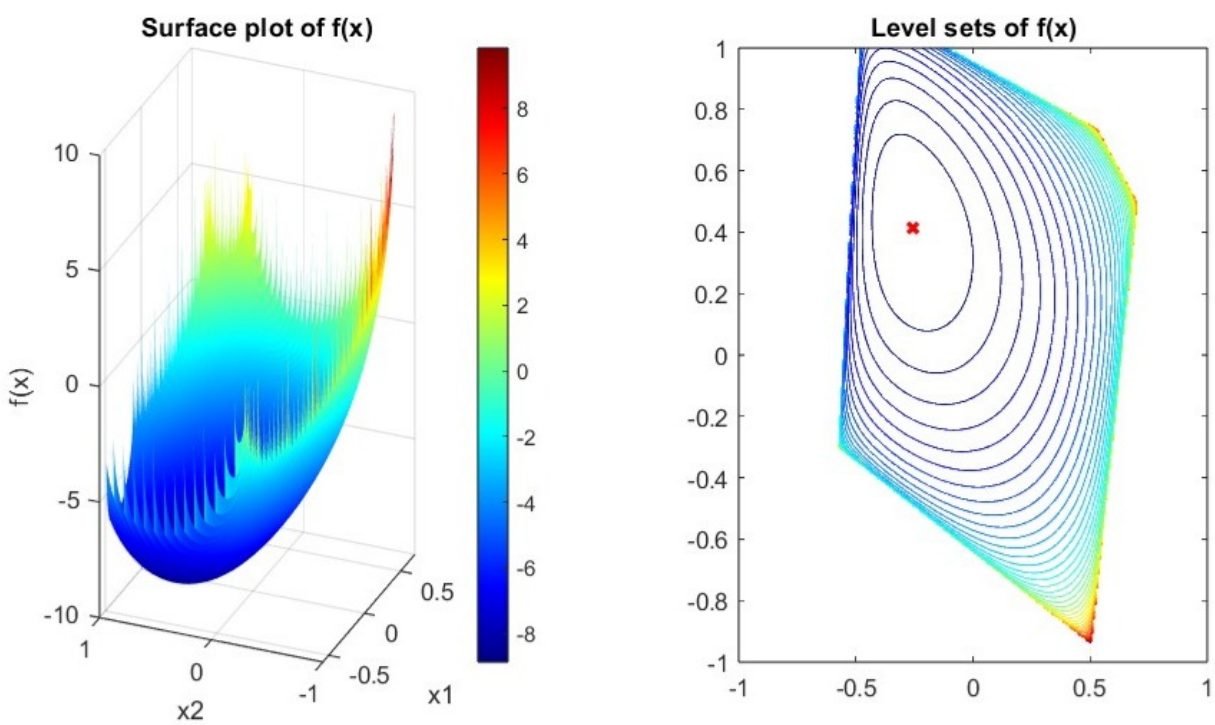
2. The CVX implementation of the problem is:

```
cvx_begin quiet
    variable x(n)
    minimize( c' * x - sum( log(b - A*x) ) )
    subject to
        b - A*x >= 1e-6;    % keep x strictly inside dom f
cvx_end
```

   Then, if there is a solution, we display the x_ref (x_ref = x) and the p_star (p_star = cvx_optval):

```
Reference optimal point x* =
    -0.2599
     0.4126

Reference optimal value p* =
    -8.8658
```

3. For the two-dimensional case (n == 2), we visualize the objective function on a regular grid.



4. We implement Gradient Descent starting from x = 0 and store the function values at each iteration:

```
alpha = 0.25;     % Armijo parameter
beta  = 0.5;      % step reduction factor
max_iters = 50;

xk = zeros(n,1);
```

```
f_grad = zeros(max_iters,1);

for k = 1:max_iters
    g = gradf(xk);    % gradient at current point
    t = 1;            % initial step length

    % (a) ensure feasibility of xk - t*g
    while ~all(b - A*(xk - t*g) > 0)
        t = beta * t;
    end

    % (b) Armijo backtracking
    while true
        x_new = xk - t*g;
        if f(x_new) <= f(xk) - alpha*t*norm(g)^2
            break;
        end
        t = beta * t;
    end

    xk = x_new;
    f_grad(k) = f(xk);
end
```

We first shrink t until the candidate point $x_k - t * g$ belongs to the domain. Then we apply the Armijo condition to guarantee sufficient decrease.

5. At iteration k, we compute:

   (a) Gradient: $g_k = \nabla f(x_k)$

   (b) Hessian: $H_k = \nabla^2 f(x_k) = A^T \operatorname{diag}\left((b - Ax_k)^{-2}\right) A$.
   which is positive definite inside dom f.

   (c) Newton direction: $\Delta x_k = -H_k^{-1} g_k$.

   Newton's Method is implemented in a similar structure in MATLAB:

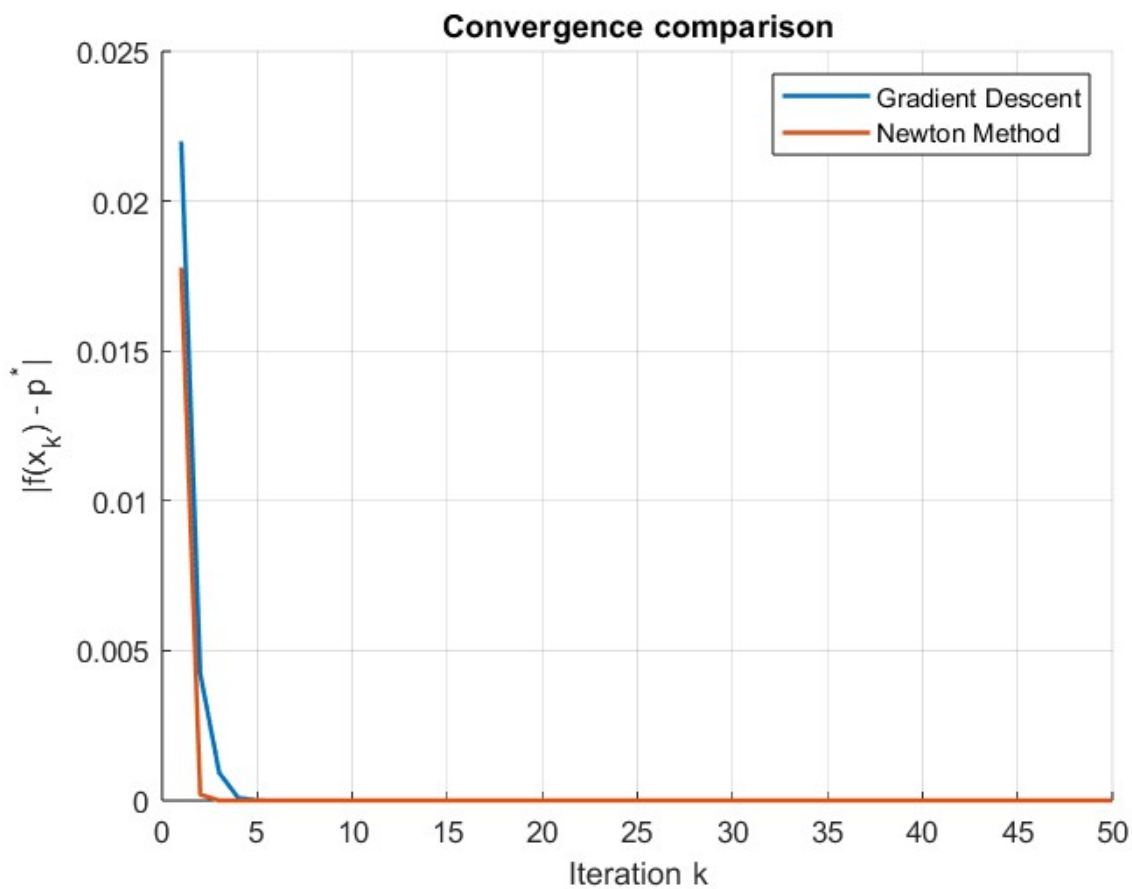
```
xk = zeros(n,1);
f_newt = zeros(max_iters,1);

for k = 1:max_iters
    g = gradf(xk);

    % Hessian matrix H = A' * diag(1./(b - A*xk).^2) * A
    D = (1 ./ (b - A*xk)).^2;
    H = A' * diag(D) * A;

    delta = - H \ g;   % Newton step
    t = 1;

    % (a) ensure feasibility of xk + t*delta
    while ~all(b - A*(xk + t*delta) > 0)
        t = beta * t;
    end

    % (b) Armijo backtracking for Newton step
    while true
        x_new = xk + t*delta;
        if f(x_new) <= f(xk) + alpha*t*(g'*delta)
            break;
        end
        t = beta*t;
    end

    xk = x_new;
    f_newt(k) = f(xk);
end
```

We enforce feasibility first, then perform a standard backtracking line search using the Newton direction. The function values are stored in $f_newt(k)$.

6. We compare Gradient Descent and Newton's Method by plotting the error $|f(x_k) - p^*|$ on a logarithmic scale:

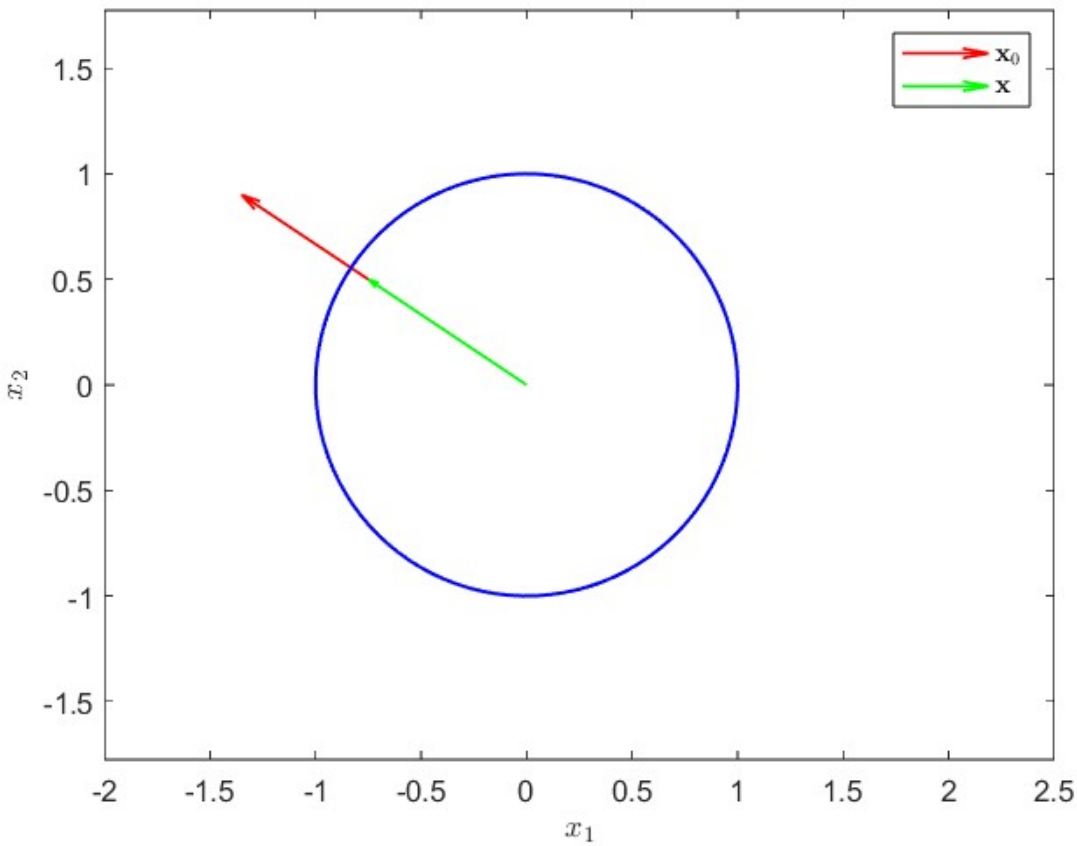
```
semilogy(abs(f_grad - p_star), 'LineWidth',1.5);
semilogy(abs(f_newt - p_star), 'LineWidth',1.5);
```

Subtracting p_{star} (the reference optimal value) and plotting with semilogy clearly shows the convergence speed:
– Gradient Descent decreases the error gradually,
– Newton's Method exhibits a much steeper drop, especially near the optimum.



2 PART B.

1. (a) First of all, we will draw a scheme of the problem in MATLAB.



(b) The equations we must solve are:

$$\text{minimize} \quad f_0(x) = \|x_0 - x\|_2^2$$

$$\text{s.t.} \quad f_1(x) = \|x\|_2^2 - r^2 \leq 0$$

(c) Gradient of f_0 :

$$\nabla f_0(x) = 2(x - x_0)$$

Gradient of f_1 :

$$\nabla f_1(x) = 2x$$

The KKT conditions are:

- $\nabla f_0(x_*) + \lambda_* \nabla f_1(x_*) = 0$

- $\lambda_* \geq 0$
- $f_1(x_*) \leq 0$
- $\lambda^* f_1(x_*) = 0$

(d) We consider the case where $\lambda_* > 0$.
Then using the first condition and solving for $\mathbf{x}_*$ we get:

$$fx_* = \frac{1}{1+\lambda_*}\mathbf{x}_0 \tag{1}$$

Then substituting to the fourth condition and solving for λ_* we get:

$$\lambda_* = \frac{\|\mathbf{x}_0\|}{r} - 1 \tag{2}$$

Using (2) equation (1) becomes:

$$fx_* = \frac{r}{\|\mathbf{x}_0\|}\mathbf{x}_0$$

This is the projection of $\mathbf{x}_0$ to the set $\mathbf{B}(\mathbf{0}, r)$ for $\lambda_* > 0$. We can see the projection in Fig. 1. We observe that if $\mathbf{x}_0 \notin \mathbf{B}(\mathbf{0}, r)$ the projection of $\mathbf{x}_0$ is the vector scaled in a way that makes it to belong to $\mathbf{B}(\mathbf{0}, r)$.

(e) Now we consider the case where $\lambda_* = 0$. For this case we solve $\nabla f_0(\mathbf{x}_*) = 0$ to get:

$$\mathbf{x}_* = \mathbf{x}_0$$

In this case it is obvious that $\mathbf{x}_0 \in \mathbf{B}(\mathbf{0}, r)$ and that is the reason the projection of $\mathbf{x}_0$ is itself.

2. Let $\mathbf{a} \in \mathbb{R}^n$. We will compute the projection of $fx_0 \in \mathbb{R}^n$ onto the set $\mathbb{S} = \{\mathbf{x} \in \mathbb{R}^n \mid \mathbf{a} \leq \mathbf{x}\}$.

The optimization problem that we have to solve becomes :

$$\begin{aligned} \text{minimize} \quad & \frac{1}{2}\|fx_0 - fx\|_2^2 \\ \text{s.t.} \quad & a_i - x_i \leq 0 \end{aligned}$$

The KKT conditions for this case are:

- $\nabla f_0(fx_*) + \sum_{i=1}^n \lambda_i \nabla f_i(fx_*) = 0$
- $\lambda_i \geq 0, \quad i \in \{1, \ldots, n\}$

- $f_i\left(fx_*\right) \leq 0, \quad i \in \{1, \ldots, n\}$
- $\lambda_i f_i\left(fx_*\right) = 0, \quad i \in \{1, \ldots, n\}$

To start off, we will compute the gradients as follows :

$$\nabla f_0(fx) = fx - fx_0$$

$$\nabla f_i(fx) = \nabla\left(fc_i^T(fa - fx)\right) = -fc_i$$

Where fc_i is the vector that its inner product with another vector fv gives us the i-th element of the last, i.e. $fc_i^T fv = v_i$.

Substituting to the first KKT condition we get :

$$fx_* - fx_0 - \sum_{i=1}^{n} \lambda_i fc_i = f0 \Rightarrow fx_* - fx_0 - \lambda = 0 \Rightarrow x_{*,i} = x_{0,i} + \lambda_i \tag{3}$$

Now we distinguish the following cases:

i. $\lambda_i = 0$
   Substituting to (5) we get $x_{*,i} = x_{0,i}$ which means that $x_0 \in S$.

ii. $\lambda_i > 0$
   Using the fourth KKT condition we find that:

$$\lambda_i = a_i - x_{0,i} \tag{4}$$

   Substituting equation (4) to (3) we get that $x_{*,i} = a_i$.

   The projection can be written in a more compact form using the function max as:

$$x_{*,i} = \max\{a_i, x_{0,i}\}$$

   or

$$x_* = \max\{a, x_0\}$$

3. Let $A \in \mathbb{R}^{p \times n}$, $b \in \mathbb{R}^p$ and $x_0 \in \mathbb{R}^n$. We will solve the following optimization problem with affine constraints:

$$\begin{array}{ll} \text{minimize} & \frac{1}{2}\|x_0 - x\|_2^2 \\ \text{subject to} & Ax = b \end{array}$$

The KKT conditions are:

- $x_* - x_0 + A^T v = 0$
- $Ax_* - b = 0$

We can re-write the conditions above in the following form:

$$\begin{bmatrix} I_{n\times n} & A^T \\ A & O \end{bmatrix} \begin{bmatrix} x_* \\ v \end{bmatrix} = \begin{bmatrix} x_0 \\ b \end{bmatrix} \tag{7}$$

The matrix

$$F = \begin{bmatrix} I_{n\times n} & A^T \\ A & O \end{bmatrix}$$

has dimensions $(n+p)\times(n+p)$ and is invertible. Thus equation (7) has a closed-form solution of the form:

$$\begin{bmatrix} x_* \\ v \end{bmatrix} = F^{-1} \begin{bmatrix} x_0 \\ b \end{bmatrix} \quad \Rightarrow \quad x_* = F^{-1}(1:n,:) \begin{bmatrix} x_0 \\ b \end{bmatrix}.$$

Notice that the projection is given by the multiplication of the first n rows of the inverse of F, which is an $n \times (n+p)$ matrix, times the vector

$$\begin{bmatrix} x_0 \\ b \end{bmatrix} \in \mathbb{R}^{n+p}.$$

4. Let $\mathbb{S} = \{\mathbf{x} \in \mathbb{R}^2 \mid \|\mathbf{x}\| \leq 1, x_1 \geq 0, x_2 \geq 0, x_1 + x_2 \leq 1\}$.
   We will solve the problem :

$$(P) \underset{\mathbf{x}\in\mathbb{S}}{\text{minimize}} f_0(\mathbf{x})$$

First of all we will draw a figure with every constraint to check if there exists a redundant constraint.

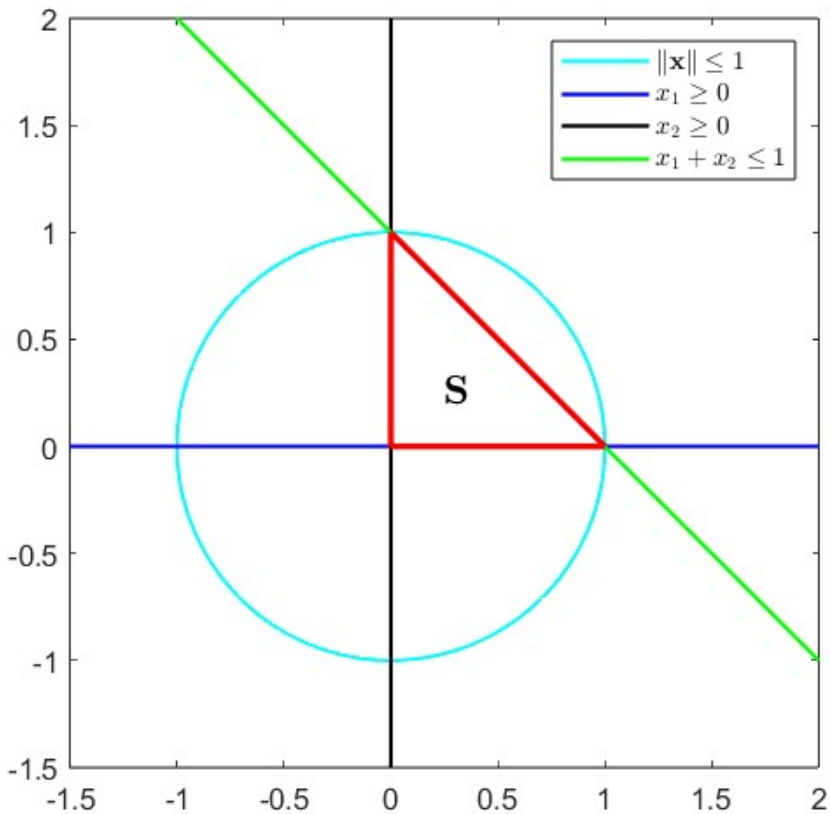


Fig. 2

We observe that the constraint $\|\mathbf{x}\| \leq 1$ is redundant.
In that case we can simplify $\mathbb{S}$ to be :

$$\mathbb{S} = \left\{\mathbf{x} \in \mathbb{R}^2 \mid x_1 \geq 0, x_2 \geq 0, x_1 + x_2 \leq 1\right\}$$

Problem (P) can be re written in the following form.

$$\begin{array}{ll} \text{minimize} & f_0(\mathbf{x}) \\ \text{s.t.} & -x_1 \leq 0 \\ & -x_2 \leq 0 \\ & x_1 + x_2 \leq 1 \end{array}$$

(a) $f_0(x) = (x_1 - 2)^2 + (x_2 - 2)^2$

The KKT conditions are:

- $\nabla f_0(\mathbf{x}_*) + \sum_{i=1}^{3} \lambda_i \nabla f_i(\mathbf{x}_*) = 0$
- $\lambda_i \geq 0, i \in \{1, \ldots, n\}$
- $f_i(\mathbf{x}_*) \leq 0, i \in \{1, \ldots, n\}$
- $\lambda_i f_i(\mathbf{x}_*) = 0, i \in \{1, \ldots, n\}$

Where the gradients from above are:

$$\nabla f_0(\mathbf{x}) = \begin{bmatrix} \frac{\partial f_0}{\partial x_1} \\ \frac{\partial f_0}{\partial x_2} \end{bmatrix} = \begin{bmatrix} 2x_1 - 4 \\ 2x_2 - 4 \end{bmatrix}$$

All 3 constraints are of the form $f_i(\mathbf{x}) = \mathbf{c}_i^T \mathbf{x} + b_i \Rightarrow \nabla f_i(\mathbf{x}) = \mathbf{c}_i$.
Thus the gradients of the constraints are :

$$\nabla f_1(\mathbf{x}) = \begin{bmatrix} -1 \\ 0 \end{bmatrix}, \quad \nabla f_2(\mathbf{x}) = \begin{bmatrix} 0 \\ -1 \end{bmatrix}, \quad \nabla f_3(\mathbf{x}) = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Using equations and substituting to the first KKT condition, we get the following system of equations :

$$\begin{aligned} 2x_1 - 4 - \lambda_1 + \lambda_3 = 0 \Rightarrow \quad & x_1 = \frac{\lambda_1 - \lambda_3 + 4}{2} \\ 2x_2 - 4 - \lambda_2 + \lambda_3 = 0 \quad & x_2 = \frac{\lambda_2 - \lambda_3 + 4}{2} \end{aligned}$$

The second pair of equations comes from the fourth KKT condition

$$\begin{aligned} \lambda_1 x_1 &= 0 \\ \lambda_2 x_2 &= 0 \end{aligned}$$

Now we distinct the following cases.

- For $\lambda_3 = 0$
  ($f_3(\mathbf{x}) \leq 0$ is not an active constraint)
  That means that our values x_1 and x_2 become :

  $$x_1 = \frac{\lambda_1 + 4}{2}, \quad x_2 = \frac{\lambda_2 + 4}{2}$$

  Now we will examine the different scenarios for λ_1 and λ_2.

  i. $\lambda_1 = \lambda_2 = 0$
     That means that $x_1 = x_2 = 2$. From the third KKT condition we have :

$$x_1 + x_2 - 1 \leq 0 \Rightarrow 2 + 2 - 1 = 3 \leq 0$$

That means that the constraints can not be all inactive at the same time, by contradiction.

ii. $\lambda_1 > 0$ and $\lambda_2 = 0$
$x_1 = 0 \Rightarrow \lambda_1 = -4 < 0$, contradiction.
This can not be the case.

iii. $\lambda_1 = 0$ and $\lambda_2 > 0$
Likewise $x_2 = 0 \Rightarrow \lambda_2 = -4 < 0$. This is not the case either.

iv. $\lambda_1 > 0$ and $\lambda_2 > 0$
For the same reason as (ii) and (iii) this case is not acceptable.

That means that $f_3(\mathbf{x}) \leq 0$ is an active constraint, meaning that the optimal lies on this line.

- For $\lambda_3 > 0$ we distinguish the same cases.

  i. $\lambda_1 = \lambda_2 = 0$ From the fourth KKT condition we derive that $x_1 + x_2 = 1$. The values in this case are :

  $$x_1 = \frac{\lambda_3 + 4}{2}, \quad x_2 = \frac{\lambda_3 + 4}{2}$$

  The above equations allow us to find that $x_1 = x_2 = \frac{1}{2}$ for $\lambda_1 = \lambda_2 = 0$, $\lambda_3 = 3$, which all satisfy the KKT conditions meaning that the optimal point is:

  $$x_* = \begin{bmatrix} \frac{1}{2} \\ \frac{1}{2} \end{bmatrix}$$

  ii. $\lambda_1 = 0$ and $\lambda_2 > 0$

  $$x_1 = \frac{(-\lambda_3 + 4)}{2}, x_2 = \frac{\lambda_2 - \lambda_3 + 4}{2} = 0$$

  and

  $$x_1 = 1 - x_2 = 1 \Rightarrow \lambda_3 = 2$$

  This means that $\lambda_2 = \lambda_3 - 4 = -2$ which is a contradiction with the second KKT condition.

  iii. $\lambda_1 > 0$ and $\lambda_2 = 0$
  Likewise with (ii) $\lambda_3 = 2 \Rightarrow \lambda_1 = -2$ we result in a contradiction.

iv. $\lambda_1 > 0$ and $\lambda_2 > 0$ This means that $x_1 = x_2 = 0$. Substituting to $x_1 + x_2 = 1 \Rightarrow 0 = 1$ which is a contradiction.

(b) $f_0(x) = (x_1 + 2)^2 + (x_2 + 2)^2$

For this cost function the KKT conditions are the same with the previous one but here the gradient of f_0 is:

$$\nabla f_0(\mathbf{x}) = \begin{bmatrix} \frac{\partial f_0}{\partial x_1} \\ \frac{\partial f_0}{\partial x_2} \end{bmatrix} = \begin{bmatrix} 2x_1 + 4 \\ 2x_2 + 4 \end{bmatrix}$$

Using the KKT conditions we get the following system of equations:

$$x_1 = \frac{\lambda_1 - \lambda_3 - 4}{2}$$

$$x_2 = \frac{\lambda_2 - \lambda_3 - 4}{2}$$

$$\lambda_1 x_1 = 0$$

$$\lambda_2 x_2 = 0$$

$$\lambda_3(x_1 + x_2 - 1) = 0$$

Following the same methodology we distinguish 2 cases for λ_3.

- For $\lambda_3 > 0$, we have:

$$x_1 + x_2 = 1$$

i. $\lambda_1 = \lambda_2 = 0$

$$x_1 = x_2 = \frac{-\lambda_3 - 4}{2} \Rightarrow -\lambda_3 - 4 = 1 \Rightarrow \lambda_3 = -3$$

This can not be the case where the optimal point occurs since λ_3 must be positive.

ii. $\lambda_1 > 0$ and $\lambda_2 = 0$

$$x_1 = 0 \Rightarrow x_2 = \frac{-\lambda_3 - 4}{2} = 1 \Rightarrow \lambda_3 = -2$$

For the same reason as in case (i) this case is not acceptable.

iii. $\lambda_1 = 0$ and $\lambda_2 > 0$

$$x_2 = 0 \Rightarrow x_1 = \frac{-\lambda_3 - 4}{2} = 1 \Rightarrow \lambda_3 = -2$$

iv. $\lambda_1 > 0$ and $\lambda_2 > 0$

$$x_1 = x_2 = 0$$

The equation $x_1 + x_2 = 1$ is not satisfied.

That means that the optimal point occurs when $\lambda_3 = 0$ which means that the third constraint is inactive.

- For $\lambda_3 > 0$ we have:

$$x_1 = \frac{\lambda_1 - 4}{2}, x_2 = \frac{\lambda_2 - 4}{2}$$

i. $\lambda_1 = \lambda_2 = 0$

$$x_1 = x_2 = -2$$

That can not happen since x1 and x2 must be positive.

ii. $\lambda_1 > 0$ and $\lambda_2 = 0$

$$x_2 = -2$$

Contradiction due to the third KKT condition.

iii. $\lambda_1 = 0$ and $\lambda_2 > 0$

$$x_1 = -2$$

Likewise with (ii).

iv. $\lambda_1 > 0$ and $\lambda_2 > 0$

$$x_1 = x_2 = 0 \Rightarrow \lambda_1 = \lambda_2 = 4$$

In the last case every KKT condition is satisfied.
The optimal point and the corresponding lagrangian multipliers are :

$$x_* = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \lambda_1 = \lambda_2 = 4, \lambda_3 = 0$$

We can also observe that the active constraints are both the first and the second. Looking at Fig. 2, the only point where both of these constraints are active is the origin. Having said that, it is logical to compute that minimum point.